SOULSENSE

Another Crab's Treasure Combat & Build Advisor

A rule-based expert system with forward chaining, conflict resolution and an explanation facility. Code explanation: modules, rules, workflow, testing and deployment.

Rules in the knowledge base	95
Rule categories	8
Derived (inferred) fact types	5
User-input facts	44
Language / UI	Python 3 / Streamlit
Inference method	Forward chaining (match - resolve - act)

1. What the system does

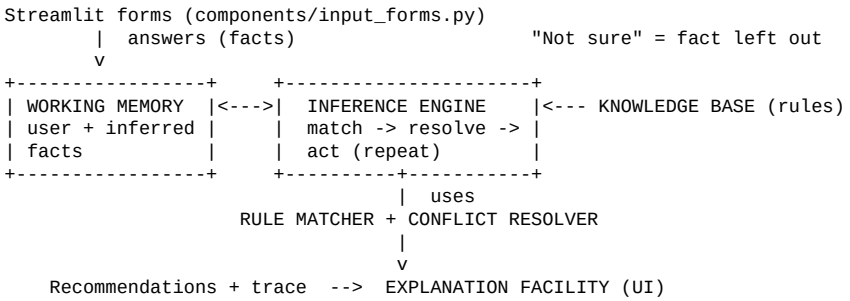
SOULSENSE is a rule-based expert system that advises players of *Another Crab's Treasure* on combat, shells, weapons and attacks, stat allocation, adaptations, healing and survival, boss preparation and defensive play. The player describes their situation through Streamlit forms. The inference engine evaluates those answers (facts) against the knowledge base (IF-THEN rules) and returns prioritised recommendations, each one explained by the rule that produced it.

It is a genuine expert system rather than a questionnaire: no answer is hard-coded to a response. Responses are the conclusions of rules, some of which depend on facts that other rules inferred. Machine learning is not used anywhere.

2. Architecture

The classic expert-system components map one-to-one onto the code:

Component	File	Responsibility
Knowledge base	knowledge_base/rules.py	95 IF-THEN rules, thresholds, rule categories
Fact vocabulary	knowledge_base/facts.py	Every user input with type, options and label
Evidence base	knowledge_base/game_mechanics.md	Verified mechanics M-01..M-24 cited by rules
Working memory	engine/working_memory.py	Stores user facts and inferred facts with their origin
Rule matcher	engine/rule_matcher.py	Evaluates AND / OR / NOT trees with 3-valued logic
Conflict resolver	engine/conflict_resolver.py	Priority, specificity, KB order; advice groups
Inference engine	engine/inference_engine.py	Forward-chaining loop, trace, missing-input analysis
Explanation facility	components/result_display.py	Shows fired rules, matched values, chains
User interface	app.py, pages/, components/	Streamlit pages, forms generated from the schema



3. Module walk-through

working_memory.py - class `WorkingMemory`. `assert_user_fact` stores a form answer (None, meaning 'not sure', is ignored). `assert_inferred` stores a rule conclusion and returns ADDED, UNCHANGED or BLOCKED. Memory is monotonic (facts are never removed or overwritten) which guarantees termination, and user facts always beat inferred ones.

rule_matcher.py - conditions are a small tree: `Cond(fact, operator, value)` leaves combined with `AllOf` (AND), `AnyOf` (OR) and `Not`. Operators: == != < <= > >= in, not in, contains, not contains, contains any, is true, is false, count ==, count >=, count <. Evaluation returns True, False or None (unknown) plus explanation lines that show the actual value of every fact tested.

rule.py - the `Rule` dataclass: id, category, name, conditions, priority, advice, asserts (inferred facts), severity, group, basis, sources and rationale.

conflict_resolver.py - ranks matched rules and resolves contradictory advice (section 5).

inference_engine.py - class `InferenceEngine`; `run(user_facts)` executes forward chaining and returns an `InferenceResult` containing every firing, the recommendations, the overruled advice, rules that could not be evaluated and the final working memory.

rule_analysis.py - a small constraint solver that builds an input satisfying any rule. It proves that no rule is dead and supplies the 'example input' shown in the Knowledge Explorer.

components/ - `session.py` (shared answers dictionary and engine access), `input_forms.py` (widgets generated from the fact schema), `result_display.py` (cards, traces, explanations), `navigation.py` (st.navigation registry).

pages/ - one module per screen, each exposing `render()`: home, profile, combat_advisor, shell_advisor, boss_advisor, progression_advisor, diagnosis, knowledge_explorer.

4. Forward chaining

Forward chaining is data-driven: begin with known facts and keep firing rules until nothing new follows. Each cycle has three steps:

1. Match - every rule that has not fired yet is evaluated; those whose conditions are TRUE form the *conflict set*.
2. Resolve - the conflict resolver picks one rule.
3. Act - the rule fires: it asserts its inferred facts and/or records its advice. The loop repeats because new facts may make further rules match. It stops when the conflict set is empty. A rule can fire at most once (refraction), so the loop always terminates.

Chaining example. Rule SHL-01 (priority 100) infers `shell_condition = critical` from `shell_durability_pct <= 30`. Only after that can SHL-04 match on `shell_condition == 'critical'` and advise swapping the shell. The explanation shows this dependency as 'Chain: uses conclusions of SHL-01'.

Worked trace of the scenario *Blocker with a cracking shell* (generated by the real engine):

```
cycle  1  SHL-01  Shell is critical                       won: same priority (100); more specific (2 te  -> infers
              {'shell_condition': 'critical'}
cycle  2  SRV-03  HP is fine                              won: highest priority (100)  -> infers {'hp_state': 'ok'}
cycle  3  SHL-04  Swap a failing shell                    won: highest priority (90)
cycle  4  DEF-06  Conserve a worn shell                   won: highest priority (65)
cycle  5  DEF-07  Parry relentless combos                 won: same priority (60); more specific (4 tes
cycle  6  SHL-05  Kintsugi cushions a breaking shell      won: highest priority (60)
cycle  7  SHL-11  Light shell versus a blocking style     won: same priority (55); more specific (3 tes
cycle  8  PRG-03  RES for blockers                        won: highest priority (55)
cycle  9  DEF-08  Learn Riposte                           won: highest priority (50)
cycle 10  DEF-01  There is no stamina bar                 won: only matching rule
```

Full explanation of one recommendation as the system prints it:

```
SHL-04  Swap a failing shell   [cycle 3, priority 90, specificity 2]
  IF   (shell_condition == 'critical' AND nearby_shells is true)
       AND of:
          OK  shell_condition == 'critical'   [actual: 'critical']
          OK  nearby_shells is true   [actual: True]
  THEN  Your shell is about to break and a replacement is nearby - swap now. A broken shell means no shell spell, no
        block and much less damage reduction.
```

```
WHY   A shell breaks at 0 durability and a shell-less crab relies on dodging only.
WON   highest priority (90)  (conflict set: SHL-04, DEF-06, DEF-07, SHL-05, SHL-11, PRG-03, DEF-08, DEF-01)
CHAIN uses conclusions of SHL-01
```

5. Conflict resolution

Two kinds of conflict exist, so two documented strategies are used.

Agenda conflict - which matched rule fires next? The conflict set is sorted by (a) **priority**, higher first; (b) **specificity**, the number of elementary tests in the rule, higher first; (c) **position in the knowledge base**, earlier first. The reason that decided the choice is stored with every firing and displayed to the user.

Derivation rules use priority 100 so the facts they infer exist before advice rules are chosen. If two rules infer different values for the same fact, the first (higher-ranked) wins and the other is reported as 'blocked'.

Recommendation conflict - contradictory advice. Rules that answer the same question share a *group* (defense_mode, weight_choice, stat_priority, resource_action, heal_or_rest, boss_phase). Only the best-ranked advice of a group is shown as primary; the others appear under 'Advice overruled by conflict resolution' with the name of the winning rule, so nothing is hidden.

Group	Question answered	Example resolution
defense_mode	Block, dodge or parry?	DEF-06 (conserve a worn shell, priority 65) overrules CMB-03 (block, 50)
stat_priority	Which stat first?	PRG-02 beats PRG-01 for veteran attackers: same priority, more specific
heal_or_rest	Use a pod or rest at the Moon Snail Shell?	SRV-06 (area already cleared, 72) overrules SRV-05 (70)
boss_phase	Generic or live phase-two advice?	BOS-03 (HP below 50 percent) overrules BOS-02
weight_choice	Heavy or light shell?	Mutually exclusive playstyles, ranked if both match
resource_action	Spend or bank microplastics?	Decided by whether a checkpoint is near

6. Handling missing or uncertain input

Every form field offers 'Not sure'. Such an answer is not stored, so the fact is **unknown**. The matcher uses three-valued logic: AND is false if any part is false, unknown if any part is unknown, otherwise true; OR is true if any part is true, unknown if any part is unknown, otherwise false. A rule with an unknown condition simply does not fire, and the engine traces unknown derived facts (for example shell_condition) back to the user questions behind them. The UI then lists 'Answer more to unlock N rules' with the most useful questions first. Empty input never crashes the system.

7. Knowledge base

Rules are grouped in eight categories. Evidence types: **mechanic** (follows from a verified game mechanic), **guide** (community-guide recommendation), **tactic** (general tactic applied to a mechanic), **design** (classification written by the designer, for example 'durability up to 30 percent is critical'). Threshold values are named constants at the top of rules.py and are not claimed to be game mechanics.

Category	Rules
Combat & enemy encounters	9
Shells, durability & defense	13
Weapon & attack strategies	8
Character progression & stats	13
Adaptations & abilities	9
Healing, resources & survival	16
Boss preparation & strategy	18
Dodging, blocking, parrying & stamina	9

Where sources disagree (for example which stat to raise first) the knowledge base does not pick a side; it gives playstyle-specific advice. Where no verified boss mechanic exists, rule BOS-18 says so instead of inventing advice. Full evidence list: knowledge_base/game_mechanics.md.

7.1 Combat & enemy encounters

ID	Rule / IF	THEN	P
CMB-01	**Detect a group fight** IF enemy_count >= 3	Infers group_fight=True.	100
CMB-02	**Do not fight a group in the open** IF group_fight is true	Three or more enemies: avoid fighting in the open. Back up to a wall or choke point and prefer area attacks - the Twist Top shell spell or the Eelectrocute adaptation hit several enemies at once.	60
CMB-03	**Block against relentless attackers** IF (attack_style == 'fast_relentless' AND shell_worn is true AND shell_condition != 'critical')	Fast, relentless combos leave little room to roll. Lean on blocking (the shell absorbs the damage instead of Kril) and attack in the gaps - but swap shells before yours breaks.	50
CMB-04	**Dodge into slow, telegraphed attacks** IF attack_style in ['slow_heavy', 'area_slam']	Slow, telegraphed swings: dodge INTO the attack (invincibility frames), then punish from the sides or from behind.	45
CMB-05	**Heavy shell versus slow attacks: block or parry** IF (attack_style in ['slow_heavy', 'area_slam'] AND shell_weight == 'L')	Your heavy shell slows your dodge roll. Against slow, telegraphed attacks it is safer to block (or parry) than to roll late.	45
CMB-06	**Close the gap on ranged enemies** IF (attack_style == 'ranged' AND skills_known contains any ['spearfishing', 'wave_breaker'])	Ranged attackers: close the distance with Spearfishing (reels the enemy in, costs a Barbed Hook) or Wave Breaker (grapple and strike) instead of walking through projectiles.	55
CMB-07	**Rubber Band equipped: ignore electricity** IF (attack_style == 'electric' AND stowaways_equipped contains 'rubber_band')	Rubber Band is equipped, so you can ignore electric attacks - stay aggressive.	40
CMB-08	**Electric enemy without Rubber Band** IF (attack_style == 'electric' AND stowaways_equipped not contains 'rubber_band')	Electric attackers: if you own the Rubber Band stowaway, equip it - it lets you ignore electric attacks.	55
CMB-09	**Learn patterns after repeated deaths** IF (encounter_type == 'tough' AND deaths_in_a_row >= 2)	You keep dying to the same tough enemy. Stop trading blows blindly: watch one complete attack cycle, learn the wind-ups, then re-engage. Some common enemies are as dangerous as bosses.	50

7.2 Shells, durability & defense

ID	Rule / IF	THEN	P
SHL-01	**Shell is critical** IF (shell_worn is true AND shell_durability_pct <= 30)	Infers shell_condition=critical.	100
SHL-02	**Shell is worn** IF (shell_worn is true AND shell_durability_pct > 30 AND shell_durability_pct <= 60)	Infers shell_condition=worn.	100
SHL-03	**Shell is healthy** IF (shell_worn is true AND shell_durability_pct > 60)	Infers shell_condition=healthy.	100
SHL-04	**Swap a failing shell** IF (shell_condition == 'critical' AND nearby_shells is true)	Your shell is about to break and a replacement is nearby - swap now. A broken shell means no shell spell, no block and much less damage reduction.	90
SHL-05	**Kintsugi cushions a breaking shell** IF (shell_condition == 'critical' AND skills_known contains 'kintsugi')	You know Kintsugi: this shell regenerates fully after it breaks - once per rest. Keep using it, but remember the free regeneration is spent after one break.	60
SHL-06	**Use Elusive Prey when the shell breaks** IF (shell_condition == 'critical' AND skills_known contains 'elusive_prey')	Elusive Prey gives a couple of seconds of invincibility when your shell breaks. Use that window to reposition or heal - attacking early ends it.	60
SHL-07	**Shell-less with a shell in reach** IF (shells_unlocked is true AND shell_worn is false AND nearby_shells is true)	You are shell-less and a shell is within reach - pick it up. Without a shell you have no spell, no block and much less damage reduction.	90
SHL-08	**Summon Snail Sanctum when naked** IF (shells_unlocked is true AND shell_worn is false AND nearby_shells is false AND adaptations_known contains 'snail_sanctum' AND umami_charges >= 1)	No shell nearby: cast Snail Sanctum - when you are naked it summons a shell made of pure Umami.	85
SHL-09	**Shells are locked until Nephro** IF shells_unlocked is false	Wearing shells unlocks when you beat Nephro, the first boss. Until then you fight shell-less: rely on dodging and level up a few times first.	70
SHL-10	**Heavy shell versus a dodge-based style** IF (shell_weight == 'L' AND playstyle == 'dodger')	A heavy shell slows your dodge roll, which clashes with an evasive style. Prefer a light or medium shell, or learn Skedaddle (a backward dash not affected by shell weight).	55
SHL-11	**Light shell versus a blocking style** IF (shell_weight == 'S' AND playstyle == 'blocker' AND encounter_type in ['tough', 'boss'])	Light shells are fragile. If you mostly block, a heavier shell holds more durability against hard hitters, at the price of a slower dodge roll.	55
SHL-12	**Protect a favourite shell with insurance** IF (shell_favorite is true AND shell_insured is false)	You want to keep this shell - buy Shell Insurance. An insured shell comes back with you after death (prices vary, e.g. 200 for Banana Peel, 1,500 for Yoccult).	65
SHL-13	**Use a stagger spell on tough enemies** IF (shell_spell_role == 'stagger' AND umami_charges >= 1 AND encounter_type in ['tough', 'boss'])	Your shell spell builds stagger (e.g. Fizzle costs 1 Umami and raises stagger). Use it to fill the yellow bar, then burst the stunned enemy.	60

7.3 Weapon & attack strategies

ID	Rule / IF	THEN	P
WPN-01	**Burst when stagger is nearly full** IF stagger_pct >= 80	The stagger bar is nearly full: line up your strongest attacks - a full bar stuns the enemy briefly and that is your main damage window.	70
WPN-02	**Dispatch a capsized enemy** IF (enemy_capsized is true AND skills_known contains 'dispatch')	Capsized enemy and you know Dispatch: attack its weak spots for maximum damage.	75
WPN-03	**Capsized without Dispatch** IF (enemy_capsized is true AND skills_known not contains 'dispatch')	The enemy is capsized - take the free hits. The Dispatch skill would make weak-spot attacks on capsized enemies much stronger.	50
WPN-04	**Skedaddle into Ebb and Flow** IF (skills_known contains 'skedaddle' AND skills_known contains 'ebb_and_flow')	Combo: dodge with Skedaddle (backward dash), then Ebb and Flow gives an immediate powerful spin attack - a built-in dodge-and-counter.	50
WPN-05	**Block then counter with Aggravation** IF (skills_known contains 'aggravation' AND playstyle in ['blocker', 'balanced'] AND shell_condition != 'critical')	Aggravation: when your shell takes a blocked hit your next attack hits harder - block, then counter-attack straight away.	50
WPN-06	**Whelk stowaway for spell builds** IF (playstyle == 'umami_caster' AND stowaways_equipped not contains 'whelk' AND res >= 2)	Whelk-family stowaways raise shell-spell damage (+15 / +30 / +45%, needing 2 / 7 / 13 RES). You meet the first requirement - equip one if you own it.	45
WPN-07	**Spearfishing is weak against bosses** IF (skills_known contains 'spearfishing' AND encounter_type == 'boss')	Spearfishing struggles with large enemies and each throw uses a Barbed Hook - do not rely on it against bosses.	30
WPN-08	**Bubble Bullet capsize combo** IF (adaptations_equipped contains 'bubble_bullet' AND skills_known contains 'dispatch')	An upgraded (Lv. 2) Bubble Bullet can capsize an enemy - follow up with Dispatch on the weak spots.	45

7.4 Character progression & stats

ID	Rule / IF	THEN	P
PRG-01	**Early VIT base** IF (vit < 5 AND game_stage == 'early')	Raise VIT to at least 5 first (+5 HP per point). Early bosses hit hard and Kril has little HP.	60
PRG-02	**Veteran attackers favour ATK** IF (experience == 'veteran' AND playstyle == 'aggressor' AND game_stage == 'early')	As an experienced attacker, favour ATK (about +7% strike damage per point) with a little VIT - faster kills mean fewer hits taken.	60
PRG-03	**RES for blockers** IF (playstyle == 'blocker' AND res < 5)	Blocking sends damage into the shell. RES lowers the damage your shell takes and unlocks stowaways such as Whelk - it pays off most for blockers.	55
PRG-04	**ATK once you have a VIT base** IF (playstyle == 'aggressor' AND vit >= 5)	With a solid VIT base, further points in ATK raise strike damage by about 7% each.	50
PRG-05	**Umami Training for casters** IF (playstyle == 'umami_caster' AND skills_known not contains 'umami_training' AND umami_crystals >= 1)	Spend crystals on Umami Training (+5 MSG each, three copies) at the Moon Snail.	55
PRG-06	**Fix 'I take too much damage'** IF main_problem == 'too_much_damage'	Taking too much damage: add VIT for HP and RES for shell durability, and check the DEF and weight of the shell you wear.	65
PRG-07	**Fix 'I deal too little damage'** IF main_problem == 'too_little_damage'	Dealing too little damage: put points into ATK (about +7% per point) and work the stagger bar to open damage windows.	65
PRG-08	**Fix 'my shells break too fast'** IF main_problem == 'shells_breaking'	Shells breaking too fast: RES lowers shell damage, well-timed parries negate it, Self-Repair and Kintsugi restore shells, heavier shells hold more durability, and insurance protects favourites.	65
PRG-09	**Spend a big haul at a checkpoint** IF (carrying_much is true AND near_moon_snail is true)	You carry a lot of microplastics and a Moon Snail Shell is close - level up or spend them now. Everything you carry is dropped when you die.	85
PRG-10	**Bank a big haul safely** IF (carrying_much is true AND near_moon_snail is false)	Lots of microplastics and no checkpoint nearby: head back and bank them, and skip optional fights. Dying drops everything, and dying again before recovering it loses it for good.	85
PRG-11	**Respec with a Shark Egg** IF (unhappy_with_build is true AND shark_eggs >= 1)	Respec at any Moon Snail Shell: Level Up, then the recycle icon. It costs 1 Shark Egg and refunds the microplastics you spent.	50
PRG-12	**Buy a Shark Egg to respec** IF (unhappy_with_build is true AND shark_eggs == 0 AND microplastics >= 2000)	No Shark Egg yet: one is reported on sale in New Carcinia (Upper Crust) for about 2,000 microplastics - check the shop. Eggs are limited, so respec only if you must.	45
PRG-13	**Upgrade stowaway capacity** IF (stowaway_capacity_free is false AND microplastics >= 2500)	Your stowaway capacity is full: vendors in New Carcinia sell capacity upgrades (the first reportedly costs 2,500 microplastics).	40

7.5 Adaptations & abilities

ID	Rule / IF	THEN	P
ADP-01	**Carry two adaptations** IF (count(adaptations_known) >= 2 AND count(adaptations_equipped) < 2)	You know two or more adaptations but carry fewer than two - equip a second one (you can carry two at once).	60
ADP-02	**Out of Umami** IF (umami_charges == 0 AND count(adaptations_equipped) >= 1)	Out of Umami: adaptations and shell spells need charges. Recover them with Riposte (after a parry) or Circle of Life (kill with an Umami attack) - and fight with your fork meanwhile.	55
ADP-03	**Royal Wave before a burst** IF (adaptations_equipped contains 'royal_wave' AND encounter_type == 'boss')	Royal Wave makes enemies briefly take increased damage - cast it just before your strongest attacks or a stagger window.	45
ADP-04	**Eelectrocute for groups** IF (group_fight is true AND adaptations_equipped contains 'eelectrocute')	Eelectrocute shocks nearby enemies - exactly what a group fight needs.	50
ADP-05	**Upgrade adaptations at Topoda** IF (defeated_topoda is true AND umami_crystals >= 25 AND count(adaptations_known) >= 1)	Grovekeeper Topoda can upgrade your adaptations (Lv. 2 reportedly costs 25 crystals, Lv. 3 costs 50).	40
ADP-06	**Adaptation upgrades need Topoda** IF (defeated_topoda is false AND count(adaptations_known) >= 1 AND game_stage in ['mid', 'late'])	Adaptations are upgraded by Grovekeeper Topoda, an optional boss reachable after you beat Heikea.	25
ADP-07	**Housewarming rewards shell swapping** IF (playstyle == 'umami_caster' AND skills_known not contains 'housewarming' AND umami_crystals >= 1)	Learn Housewarming: each shell's spell can be cast once for free, and boss arenas contain plenty of shells to swap into.	45
ADP-08	**Shelleportation for banking** IF (carrying_much is true AND skills_known not contains 'shelleportation' AND umami_crystals >= 1)	Shelleportation teleports between Moon Snail Shells - the safest way to bank a big microplastic haul.	50
ADP-09	**Skills that recover shells** IF (main_problem == 'shells_breaking' AND skills_known not contains 'self_repair' AND umami_crystals >= 1)	Learn Self-Repair (lost shell health returns as you deal damage) and Kintsugi (full regeneration after a break, once per rest).	55

7.6 Healing, resources & survival

ID	Rule / IF	THEN	P
SRV-01	**HP is low** IF player_hp_pct <= 35	Infers hp_state=low.	100
SRV-02	**HP is hurt** IF (player_hp_pct > 35 AND player_hp_pct <= 60)	Infers hp_state=hurt.	100
SRV-03	**HP is fine** IF player_hp_pct > 60	Infers hp_state=ok.	100
SRV-16	**Detect a big microplastic haul** IF microplastics >= 5000	Infers carrying_much=True.	100
SRV-04	**Heal with a pod away from a checkpoint** IF (hp_state == 'low' AND heartkelp_pods >= 1 AND near_moon_snail is false)	HP is low and no checkpoint is close: use a Heartkelp pod (one button) at your next safe moment.	80
SRV-05	**Rest at a nearby Moon Snail Shell** IF (hp_state in ['low', 'hurt'] AND near_moon_snail is true)	A Moon Snail Shell is close: resting heals you, refills every Heartkelp pod and saves progress - but it respawns common enemies.	70
SRV-06	**Heal instead of resting to keep a cleared area** IF (hp_state in ['low', 'hurt'] AND near_moon_snail is true AND heartkelp_pods >= 1 AND tough_enemies_cleared is true)	You have cleared tough enemies here and resting would respawn them. Heal with a pod instead and keep the area quiet.	72
SRV-07	**Munch when out of pods** IF (hp_state == 'low' AND heartkelp_pods == 0 AND near_moon_snail is false AND shell_worn is true AND shell_spell_role == 'self_heal' AND shell_durability_pct >= 40)	No pods left: your shell spell (Munch) converts shell durability into HP. Use it while the shell is healthy.	75
SRV-08	**Retreat with no healing** IF (hp_state == 'low' AND heartkelp_pods == 0 AND near_moon_snail is false)	Low HP, no pods and no checkpoint near: avoid optional fights and retreat toward the nearest Moon Snail Shell, relying on dodges. An equipped Another Crab will revive you once if you die.	75
SRV-09	**Recover dropped microplastics first** IF (unrecovered_microplastics is true AND deaths_in_a_row >= 1)	Go straight to your death site and collect your microplastics. If you die before reaching them they disappear permanently.	95
SRV-10	**Consider the Assist menu** IF (deaths_in_a_row >= 8 AND open_to_assist is true)	Many deaths in a row: the Assist menu offers extra shell durability, lower enemy health, more forgiving parry timing and keeping microplastics on death. Using it is a valid choice.	30
SRV-11	**Sands Between traps** IF current_area == 'sands_between'	Sands Between: X marks on the sand are hidden traps that destroy your shell and can kill you - walk around them.	75
SRV-12	**Hunt Heartkelp Sprouts** IF heartkelp_sprouts_found < 6	Six Heartkelp Sprouts exist and each adds a pod; you have found fewer. Explore for the rest (the first is near the Mooncave in the Duchess's castle).	25
SRV-13	**Boost healing** IF (stowaways_equipped not contains 'used_bandage' AND (heartkelp_pods <= 2 OR main_problem == 'running_out_of_healing'))	Running thin on healing: the Used Bandage stowaway adds 15% healing effectiveness, and Heartkelp Sprouts add pods.	35
SRV-14	**Earn more microplastics** IF main_problem == 'low_microplastics'	Short on microplastics: a Fruit Sticker stowaway raises kill rewards, and farming near a Moon Snail Shell lets you bank quickly.	30
SRV-15	**Another Crab as a safety net** IF (stowaways_equipped not contains 'another_crab' AND microplastics >= 3000 AND deaths_in_a_row >= 3)	Another Crab (reported cost 3,000 microplastics) revives you once when you die, then breaks - a cushion for hard fights.	35

7.7 Boss preparation & strategy

ID	Rule / IF	THEN	P
BOS-01	**Detect a phase-change boss** IF target_boss in ['diseased_lichenthrope', 'heikea', 'voltai']	Infers boss_phase_change=True.	100
BOS-02	**Prepare for the 50% phase change** IF boss_phase_change is true	This boss gets stronger below 50% health. Keep a spare shell within reach and Heartkelp pods in reserve for phase two.	60
BOS-03	**Phase two has started** IF (boss_phase_change is true AND enemy_hp_pct <= 50)	Phase two has started - expect harder attacks. Heal when it is safe and replace a worn shell now.	70
BOS-04	**Boss arenas hold shells** IF encounter_type == 'boss'	Boss arenas usually contain plenty of shells - replace a broken one mid-fight. If you die, most bosses let you respawn at a nearby Moon Jelly instead of the distant checkpoint.	30
BOS-05	**Nephro** IF target_boss == 'nephro'	Nephro swings a big sword slowly: read the wind-up, dodge, and hit from the sides or from behind. You fight without a shell (they unlock when he falls), so level up a few times first.	60
BOS-06	**Magista** IF target_boss == 'magista'	Magista: players report success with the Soda Can shell and by staggering her. Beating her gives Royal Wave.	55
BOS-07	**Royal Shellsplitter needs a shell** IF (target_boss == 'royal_shellsplitter' AND shells_unlocked is false)	Royal Shellsplitter hits very hard against a shell-less crab. Beat Nephro first to unlock shell-wearing.	70
BOS-08	**Heikea rewards aggression** IF (target_boss == 'heikea' AND playstyle in ['blocker', 'dodger'])	Heikea's fight needs an aggressive approach: stay close and keep up the pressure rather than turtling. Beating him is also required before you can reach Topoda.	55
BOS-09	**Learn Parry before Topoda** IF (target_boss == 'topoda' AND skills_known not contains 'parry')	Topoda's fight teaches parrying - learn Parry (and ideally Riposte) from the Moon Snail first.	70
BOS-10	**Mantis Punch for the Consortium** IF (target_boss == 'consortium' AND adaptations_known not contains 'mantis_punch')	You need Mantis Punch (from Topoda) to smash the Purple Qube that blocks the way to The Consortium.	75
BOS-11	**Ceviche Sisters are a group fight** IF target_boss == 'ceviche_sisters'	Two shrimp attack at once and constantly: treat it as a group fight, focus one target and keep moving.	55
BOS-12	**Voltai** IF target_boss == 'voltai'	Voltai attacks from four holes with electronic devices; below 50% the arena fills with trash and poison, giving you less room. The Rubber Band stowaway ignores her electric attacks.	55
BOS-13	**Petroch's rolling attack** IF target_boss == 'petroch'	Petroch conjures crystals and has a devastating rolling attack: learn its timing and dodge the roll.	50
BOS-14	**Heavy shell against Petroch** IF (target_boss == 'petroch' AND shell_weight == 'L')	A heavy shell makes Petroch's roll harder to dodge - consider a lighter shell for this fight.	60
BOS-15	**Inkerton mixes ranges** IF target_boss == 'inkerton'	Inkerton mixes short-, medium- and long-range attacks: arrive with full pods and a fresh shell. Spearfishing or Wave Breaker can close the gap if you know them.	50
BOS-16	**Long multi-phase fights** IF (target_boss in ['camtscha', 'praya_dubia', 'firth'] AND heartkelp_pods <= 3)	Multi-phase boss and you carry few Heartkelp pods: rest at a Moon Snail Shell to refill them first, and look for the remaining Heartkelp Sprouts.	65
BOS-17	**Repeated boss deaths** IF (encounter_type == 'boss' AND deaths_in_a_row >= 4)	Several deaths in a row: watch one full attack cycle, swap to a shell or stowaway that counters the boss, and consider the Assist menu if the fight stops being fun.	50
BOS-18	**Bosses without detailed notes** IF target_boss in ['pagurus', 'diseased_lichenthrope', 'curdled_carbonara_connoisseur', 'scuttling_sludge_steamroller', 'roland']	This knowledge base has no verified boss-specific mechanics for this boss - the general boss, shell and survival rules above apply.	10

7.8 Dodging, blocking, parrying & stamina

ID	Rule / IF	THEN	P
DEF-01	**There is no stamina bar** IF experience in ['some', 'veteran']	Unlike most Soulslikes there is no stamina bar: attacking and dodging do not drain a meter, so do not hold back. Manage shell durability instead.	20
DEF-02	**Shell-less: dodge into attacks** IF (shells_unlocked is true AND shell_worn is false AND nearby_shells is false)	Shell-less with none in reach: you cannot block, so dodge INTO attacks (i-frames) and look for a shell quickly.	70
DEF-03	**Parry without a shell** IF (shell_worn is false AND skills_known contains 'natural_defenses')	Natural Defenses lets you parry even without a shell - release block just as the attack lands.	50
DEF-04	**Learn Parry** IF (playstyle == 'blocker' AND skills_known not contains 'parry' AND umami_crystals >= 1)	Learn Parry at the Moon Snail: a well-timed parry negates the damage a plain block would deal to your shell.	60
DEF-05	**Parry timing help** IF (skills_known contains 'parry' AND parry_trouble is true)	Parry means releasing block just as the attack lands, not holding it. Practise on slow attackers; the Assist menu can widen parry timing and add i-frames.	60
DEF-06	**Conserve a worn shell** IF (shell_condition in ['worn', 'critical'] AND playstyle == 'blocker')	Your shell is worn and you rely on blocking, which sends all damage into it. Mix in dodges and parries to conserve durability.	65
DEF-07	**Parry relentless combos** IF (attack_style == 'fast_relentless' AND skills_known contains 'parry' AND playstyle in ['blocker', 'balanced'] AND shell_worn is true)	Against fast combos a well-timed parry costs your shell nothing, where blocking every hit would wear it down. Parry the finishers and counter.	60
DEF-08	**Learn Riposte** IF (skills_known contains 'parry' AND skills_known not contains 'riposte' AND umami_crystals >= 1)	Learn Riposte: after a parry you counter-attack and regain 1 Umami charge.	50
DEF-09	**Learn Skedaddle** IF (playstyle == 'dodger' AND skills_known not contains 'skedaddle' AND umami_crystals >= 1)	Learn Skedaddle: a quick backward dash that ignores shell weight, and the start of the Ebb and Flow combo.	45

8. User interface and workflow

Page	Purpose
Home	Dashboard, rule counts, how it works, load an example player
Player profile	Questionnaire shared by every advisor
Combat advisor	Encounter-level tactics (combat, weapon and defense rules)
Shell and equipment	Durability, weight, insurance, spells, stowaways
Boss preparation	Boss-specific and phase advice
Progression and survival	Stats, skills, economy, healing, hazards
Expert diagnosis	All recommendations, forward-chaining trace, working memory, conflicts, every fired rule
Knowledge base	Browse and search all rules, facts, verified mechanics; example input per rule

Workflow: (1) widgets write answers into one shared dictionary in st.session_state; (2) an advisor page calls `run_engine()`, which runs forward chaining on that dictionary; (3) the result is rendered as cards sorted by severity, each with a 'Why this advice?' panel. Forms are generated from the fact schema, so adding a fact needs no new UI code. Using st.navigation turns off Streamlit's automatic pages/ scanning so the sidebar is exactly the structure registered in navigation.py.

9. Testing

Tests use only the standard library (`python -m unittest discover -s tests -v`). Beyond ordinary unit tests, knowledge-base integrity tests catch mistakes automatically:

Test file	What it proves
test_engine.py	Memory rules, every operator, 3-valued logic, conflict ordering, group suppression, chaining across 2 and 3 steps, refraction, blocked inference, missing-input tracing
test_rules.py	At least 50 rules, unique ids, every category populated, only known facts and valid option values used (typo guard), every derived fact is inferred by some rule, every cited mechanic exists, and every rule is reachable (an input is synthesised and the engine really fires the rule)
test_scenarios.py	Six realistic players trigger the expected rules, conflicts resolve as documented, empty and partial input are handled
test_ui_smoke.py	Every page renders for every scenario; forms do not alter stored answers; the app entry point and navigation load (AppTest when Streamlit is installed)

10. Deployment (Streamlit Community Cloud)

1. Create a public GitHub repository and push the project root (app.py at the top level).
2. Open share.streamlit.io, sign in with GitHub, choose *Create app*.
3. Select the repository, branch and set the main file path to `app.py`.
4. Deploy. Dependencies are read from requirements.txt (streamlit only).
5. Put the resulting public URL in README.md. Anyone on the internet can then open it.

11. Limitations and honesty

The knowledge was compiled from published guides, reviews and patch notes, not from the game files. Some values come from a single source and are marked as such; sources sometimes disagree and the knowledge base says so. Patches may change numbers. The advisor cannot read the game screen, so it relies on honest answers. The tests check the knowledge base for internal consistency, not the truth of the game facts - that is what the evidence document is for.

12. Likely viva questions

Question	Answer
Why is this an expert system, not an if-else script?	Knowledge (rules) is separate from the inference engine; rules are data, can chain, and the engine explains its reasoning.
Why forward chaining?	Facts arrive first (the player's situation) and we want every applicable recommendation, which is data-driven reasoning. Backward chaining suits a single goal to prove.

Question	Answer
How are conflicts resolved?	Priority, then specificity, then KB order; plus advice groups so only one answer per question is primary.
What happens with missing data?	Three-valued logic: unknown conditions do not fire; the UI says which answers would unlock more rules.
How do you know rules are not dead?	rule_analysis.py synthesises an input for each rule and a test checks the engine fires it.
How did you avoid inventing game mechanics?	Every rule cites a verified mechanic id; unverified items are left out or flagged; thresholds are labelled as designer heuristics.
Does it terminate?	Yes: monotonic memory and each rule fires at most once.